# Experiment:3

**Aim:** Practice programs using Classes and Interfaces(**Circle and Rectangle (Geometry) Using Classes and Interface)**

**Definition:**

- **Class:** A blueprint for creating objects with attributes and methods.
- **Interface:** A set of method definitions (without implementation) that classes must implement. In Python, we use **abstract base classes (ABC)** for interfaces.

**Algorithm:**

1. Create an abstract base class `Shape` with abstract methods `area()` and `perimeter()`.
2. Define a class `Circle` implementing `Shape`:
   - Initialize with radius
   - Implement `area()` and `perimeter()`
3. Define a class `Rectangle` implementing `Shape`:
   - Initialize with length and width
   - Implement `area()` and `perimeter()`
4. Create objects of `Circle` and `Rectangle`.
5. Call `area()` and `perimeter()` on each object to display results.

**Program:**

```
from abc import ABC, abstractmethod

import math

# Interface

class Shape(ABC):

    @abstractmethod

    def area(self):

        pass

    @abstractmethod

    def perimeter(self):

        pass

# Circle class implementing Shape

class Circle(Shape):
```

Dept of AIML

```
    def __init__(self, radius):
        self.radius = radius
    def area(self):
        return math.pi * self.radius ** 2
    def perimeter(self):
        return 2 * math.pi * self.radius
# Rectangle class implementing Shape
class Rectangle(Shape):
    def __init__(self, length, width):
        self.length = length
        self.width = width
    def area(self):
        return self.length * self.width
    def perimeter(self):
        return 2 * (self.length + self.width)
# Using the classes
c = Circle(5)
r = Rectangle(4, 6)
print("Circle Area:", c.area())
print("Circle Perimeter:", c.perimeter())
print("Rectangle Area:", r.area())
print("Rectangle Perimeter:", r.perimeter())
```

**OUTPUT:**
Circle Area: 78.53981633974483
Circle Perimeter: 31.41592653589793
Rectangle Area: 24
Rectangle Perimeter: 20

Dept of AIML